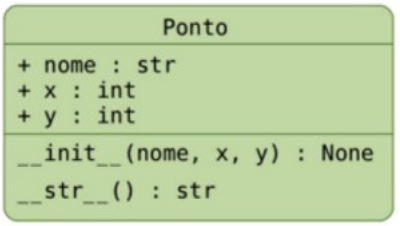
* Alguns dos Exercícios foram retirados/adaptados do curso Python Programming MOOC 2026

1) Escreva uma classe para representar uma lâmpada. Desenvolva métodos para ligar, desligar, e imprimir o status da lâmpada. Não permita, usando encapsulamento, que o status da lâmpada seja alterado fora dos métodos ligar e desligar.

2) Crie uma classe Ponto, conforme a figura a seguir. O método `__str__` deve retornar os atributos do objeto no formato "`nome: (x, y)`". Crie em outro arquivo os testes para a classe Ponto, lendo diversos pontos e criando um objeto ponto para cada entrada lida. Coloque cada objeto da classe Ponto em uma lista e, ao final, imprima cada elemento dessa lista.



3) Aproveitando a classe Ponto da questão 3, adicione o método `calcularDistancia()`, que receba um outro ponto como argumento e calcule a distância euclidiana entre eles. Considerando dois pontos (P e Q), a distância euclidiana é calculada pela fórmula:

$$d = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

Crie o programa de teste, crie alguns pontos e calcule a distância entre eles.

4) Crie uma classe Funcionário para armazenar as informações de um Funcionário. A classe deve armazenar o nome, o cargo o valor da hora trabalhada, a quantidade de horas trabalhadas e o salário. Esses dois últimos parâmetros não devem ser públicos.

- Método construtor deve receber nome, cargo e valor por hora trabalhada
  - O valor padrão da hora é 100 reais)
  - O salário e horas trabalhadas devem iniciar em zero.
- Método para registrar uma hora de trabalho.
- Método para calcular o salário.
  - Use o salário como uma propriedade.
  - O getter deve retornar o seu valor.
  - O setter deve impedir a alteração de salário de forma manual.

Teste todas as características de sua classe.

5) Crie uma classe Elevador para armazenar as informações de um elevador de prédio. A classe deve armazenar o andar atual (térreo = 0), total de andares no prédio (desconsiderando o térreo), a capacidade do elevador e quantas pessoas estão presentes nele.

A classe deve também disponibilizar os seguintes métodos:
- `Construtor:` que deve receber como parâmetros a capacidade do elevador e o total de andares no prédio (os elevadores sempre começam no térreo e vazio);
- `Entrar:` para acrescentar uma pessoa no elevador (só deve acrescentar se ainda houver espaço);
- `Sair:` para remover uma pessoa do elevador (só deve remover se houver alguém dentro dele);
- `Subir:` para subir um andar (não deve subir se já estiver no último andar);
- `Descer:` para descer um andar (não deve descer se já estiver no térreo);

Obs.: Encapsular todos os atributos da classe.

Teste todas as características de sua classe.

6) Crie uma classe chamada `Recording` que represente uma única gravação. A classe deve possuir uma variável privada: `__length`, do tipo inteiro.

Implemente os seguintes itens:

- um construtor que receba a duração da gravação como argumento;
- um método getter `length` que retorne a duração da gravação;
- um método setter que permita alterar a duração da gravação.

Deve ser possível utilizar a classe da seguinte forma:

```
the_wall = Recording(43)
print(the_wall.length)
the_wall.length = 44
print(the_wall.length)
```

Saída esperada:

```
43
44
```

Se o argumento fornecido ao construtor ou ao setter for menor que zero, deverá ser lançada uma exceção `ValueError`.

7) Crie uma classe chamada `WeatherStation`, que será utilizada para armazenar observações sobre o clima. A classe deve possuir os seguintes métodos públicos:
- um construtor que receba o nome da estação como argumento;
- um método chamado `add_observation(observation: str)` que adicione uma observação como o último elemento de uma lista;
- um método chamado `latest_observation()` que retorne a observação mais recente adicionada à lista. Se ainda não houver nenhuma observação, o método deverá retornar uma string vazia;
- um método chamado `number_of_observations()` que retorne o número total de observações adicionadas;
- um método `__str__` que retorne o nome da estação e o número total de observações adicionadas, conforme o exemplo abaixo.

Todos os atributos devem ser encapsulados, de modo que não possam ser acessados diretamente. A forma de implementar a classe fica a seu critério, desde que a interface pública seja exatamente como descrita acima.

Exemplo de utilização

```
station = WeatherStation("Houston")
station.add_observation("Rain 10mm")
station.add_observation("Sunny")
print(station.latest_observation())

station.add_observation("Thunderstorm")
print(station.latest_observation())

print(station.number_of_observations())
print(station)
```

Saída esperada

```
Sunny
Thunderstorm
3
Houston, 3 observations
```